

Chapter 1 – From Classical NLP to Text Representations

Advanced Machine Learning for Finance

Giovanni Della Lunga

University of Bologna

April-May 2026

Why This Lesson?

Motivation

The current wave of LLM applications makes it easy to use language technology without understanding it. This is useful in practice, but fragile from an intellectual and engineering point of view.

Course philosophy

We will study the field in a compact but principled way:

- ▶ enough theory to avoid black-box thinking;
- ▶ enough history to understand the transition;
- ▶ enough modern material to remain fully relevant.

Guiding idea

The central question is not *“How do I call a model?”* but rather *“How does a language system represent, compare, retrieve, and generate text?”*

Learning Objectives of the Course

By the end of the course, students should be able to:

- ▶ explain the main conceptual steps from classical NLP to LLM-based systems;
- ▶ distinguish sparse, dense, and contextual text representations;
- ▶ understand why neural networks matter for embeddings and modern language models;
- ▶ describe the logic of transformers, retrieval, and RAG;
- ▶ frame LLM applications inside broader workflows involving tools, memory, and evaluation.

Lesson 1 in the overall journey

Today we address the question: **How was text represented before transformers, and why did the field move from sparse to dense and then contextual representations?**

About This Lesson

In this lesson we will cover the following topics:

- ▶ what NLP is, and why text is a difficult kind of data;
- ▶ what a corpus is, and why domain matters;
- ▶ preprocessing as a set of choices, including subword tokenization;
- ▶ n-grams and the limits of local order modelling;
- ▶ bag-of-words, TF-IDF, Zipf's law, BM25, and cosine similarity;
- ▶ we introduce word embeddings and document embeddings;
- ▶ why all of this still matters in the age of LLMs.

Roadmap of Today's Lesson

What NLP Is and Why It Changed

What Is Natural Language Processing?

- ▶ Natural language is the medium human beings use to communicate, reason, negotiate, explain, and narrate.
- ▶ Unlike formal languages, natural language is **ambiguous**, **contextual**, **evolving**, and **domain-dependent**.
- ▶ Natural Language Processing, or **NLP**, studies how computational systems can represent, analyze, transform, and generate human language.

A first tension

Language is sequential and symbolic, but machine learning requires numerical structure. The history of NLP can be read as a sequence of increasingly sophisticated attempts to solve this tension.

Text as Data: Why Is It Hard?

Text is difficult because it combines several layers of complexity:

- ▶ **Lexical variability**: many different surface forms may refer to similar meanings.
- ▶ **Polysemy**: the same word may have different meanings in different contexts.
- ▶ **Synonymy**: different words may refer to nearby concepts.
- ▶ **Compositionality**: meaning depends on how words combine in sequences.
- ▶ **Pragmatics**: intended meaning often exceeds literal wording.

Consequence

Before generating or reasoning about text, a machine must first map language into a numerical representation where these regularities become learnable.

From Isolated Tasks to Language Systems

Classical view

NLP was often described as a collection of separate tasks:

- ▶ tokenization
- ▶ classification
- ▶ clustering
- ▶ information extraction
- ▶ sentiment analysis
- ▶ question answering

Modern view

Today, many of these tasks are addressed inside broader systems involving:

- ▶ embeddings
- ▶ retrieval
- ▶ prompting
- ▶ generation
- ▶ tool use
- ▶ memory and orchestration

From Isolated Tasks to Language Systems

Key point

The tasks did not disappear. What changed is the **representational and architectural regime** in which they are solved.

Why Classical Foundations Still Matter

- ▶ Classical NLP teaches us how text becomes data.
- ▶ It makes explicit operations that later models compress into hidden representations.
- ▶ It provides strong and interpretable baselines that remain useful in practice.
- ▶ It clarifies what is lexical, what is semantic, and what depends on context.

Without these foundations, we risk confusing

- ▶ surface overlap with meaning;
- ▶ fluent output with grounded understanding;
- ▶ retrieval with reasoning;
- ▶ and successful demos with robust systems.

Corpora, Domain, and Preprocessing

What Is a Corpus?

- ▶ A **corpus** is a collection of documents containing natural language.
- ▶ A corpus can be large or small, general or domain-specific, annotated or unannotated.
- ▶ The corpus is not merely a data container: it defines what regularities we can observe and what learning problem we can formulate.

Two broad categories

- ▶ **Annotated corpora**: useful for supervised tasks such as classification and sequence labeling.
- ▶ **Unannotated corpora**: useful for representation learning, topic discovery, clustering, and pretraining.

Example 1 — Sentiment classification

Each sentence carries a human-assigned label.
The corpus can be used to train a classifier.

Sentence	Label
<i>"The company beat earnings estimates for the fifth consecutive quarter."</i>	POS
<i>"Revenues declined 12 % year-on-year amid weak demand."</i>	NEG
<i>"The board announced no changes to the dividend policy."</i>	NEU

Annotated Corpora: Two Financial Examples

Example 2 — Named Entity Recognition (NER)

Each token is tagged with its entity type.

The corpus can be used to train a sequence labeler.

Raw sentence:

“Goldman Sachs reported Q3 net income of \$2.8 bn, up from \$2.1 bn a year earlier.”

After annotation:

Goldman Sachs_{org} reported **Q3**_{date} net income of **\$2.8 bn**_{money} , up from **\$2.1 bn**_{money} ...

■ org ■ date ■ money

Unannotated Corpora: Two Financial Examples

Example 1 — SEC 10-K filings (MD&A section)

Millions of words of raw financial prose, no labels of any kind. Useful for pretraining, topic modeling, and representation learning.

“The increase in net revenues was primarily driven by higher average loan balances and improved net interest margins, partially offset by elevated provision for credit losses reflecting continued macroeconomic uncertainty. Operating expenses increased 4 % year-over-year, largely due to higher compensation and technology investment.”

— Fictional 10-K, MD&A section, FY 2024

Example 2 — Earnings call transcript

Spoken language converted to text. Rich in hedging, forward-looking statements, and analyst questions. No labels attached.

CEO: *“We remain cautious on the second half, given persistent pressure on deposit costs. Our pipeline looks healthy and we are confident in our mid-single-digit loan growth guidance.”*

Analyst: *“Can you give more color on the NIM trajectory into Q4?”*

CFO: *“We expect NIM to stabilize around current levels, with modest upside if the Fed pauses through year-end.”*

— Fictional Q3 2024 earnings call

Why Domain Matters

- ▶ Language is deeply contextual.
- ▶ The same word can play very different semantic roles across domains.
- ▶ In finance, a *spread* is not the same thing as in epidemiology or in cooking.
- ▶ The word *bank* may refer to a financial institution, a river bank, a mine bank, or a billiard cushion.

Domain-specific corpora

Working with domain-specific corpora reduces ambiguity, narrows the prediction space, and often produces more intelligible and more robust models.

Where Do Text Corpora Come From?

Typical sources of textual corpora include:

- ▶ web pages and news feeds;
- ▶ PDF reports, filings, contracts, and manuals;
- ▶ emails, chats, tickets, and CRM interactions;
- ▶ social media posts and comments;
- ▶ structured databases with textual fields;
- ▶ speech transcripts and meeting notes.

Preprocessing Is Not Neutral

Preprocessing is often introduced as a standard pipeline, but in practice it depends on the task.

Typical operations

- ▶ tokenization
- ▶ lowercasing
- ▶ punctuation removal
- ▶ stop-word removal
- ▶ stemming
- ▶ lemmatization

Why caution is needed

- ▶ numbers may carry signal;
- ▶ punctuation may encode structure;
- ▶ stop-words may affect sentiment;
- ▶ case may matter for names and entities.

- ▶ Tokenization is the process of segmenting text into smaller units called **tokens**.
- ▶ Tokens may be words, subwords, characters, or even punctuation symbols depending on the representation model.
- ▶ In classical NLP, tokenization typically splits text into sentences and words.

Example

Text: The cat sat on the mat.

Tokens = (the, cat, sat, on, the, mat, .)

Beyond Word Boundaries: Subword Tokenization

Classical tokenization produces word-level tokens by splitting on whitespace and punctuation. Two structural problems arise immediately:

- ▶ **Out-of-vocabulary (OOV) words:** rare or technical terms unseen during training are reduced to a generic [UNK] token, losing all information.
- ▶ **Vocabulary explosion:** inflected and compound forms generate many distinct dictionary entries.

Beyond Word Boundaries: Subword Tokenization

Financial text is particularly affected

Terms such as non-performing, EBITDA, or rebalancing are rare in general corpora and are frequently classified as unknown by word-level tokenizers.

The response: subword tokenization

Split words into reusable fragments. Any word can be decomposed into known pieces – at worst into individual characters – while keeping the vocabulary small and fixed.

Subword Tokenization: BPE, WordPiece, SentencePiece

Byte Pair Encoding (BPE)

Start from characters. Iteratively merge the most frequent adjacent symbol pair until the target vocabulary size is reached.

under | val | uation

GPT-2 and GPT-4 use BPE with $\sim 50\,000$ tokens.

Variants in practice

- ▶ **WordPiece** (BERT): non-initial fragments prefixed with ##.
- ▶ **SentencePiece**: language-agnostic; treats raw bytes.
- ▶ **Unigram LM** (XLNet): probabilistic segmentation.

Forward reference – Lesson 3

Every transformer model uses subword tokenization. The tokenizer choice shapes how financial terms, tickers, and numeric expressions enter the model.

BPE: The main idea

Byte Pair Encoding

BPE builds a vocabulary of frequent **character sequences** by repeatedly merging the **most common adjacent pair** in the training data.

Very simple workflow

- 1 Start from characters (or bytes).
- 2 Count all adjacent pairs.
- 3 Merge the most frequent pair into a new symbol.
- 4 Repeat many times.

At the end, words are represented as a sequence of learned pieces such as:

`lower = low + er`

BPE: The main idea

Suppose our training corpus contains these words:

low, lower, lowest, newer

Initially we split every word into characters:

low $\rightarrow$ l o w
lower $\rightarrow$ l o w e r
lowest $\rightarrow$ l o w e s t
newer $\rightarrow$ n e w e r

Now we count adjacent pairs such as: (l,o), (o,w), (w,e), (e,r), (e,s).

BPE: The main idea

Assume the most frequent pairs are found in this order.

Merge	Interpretation
l o $\rightarrow$ lo	create token lo
o w $\rightarrow$ ow	create token ow
lo w $\rightarrow$ low	create token low
e r $\rightarrow$ er	create token er
e s t $\rightarrow$ est	create token est

Then some words can be encoded as:

lower $\rightarrow$ low + er

lowest $\rightarrow$ low + est

newer $\rightarrow$ n + ew + er or similar, depending on learned merges

Tokenization of Financial Text: A Concrete Example

How does a BPE tokenizer (GPT-4 class) segment typical financial inputs?

Input	Subword tokens
AAPL	AA PL
\$12,345.67	\$ 12 , 345 . 67
non-performing	non - performing
10-K	10 - K
EBITDA	EB IT DA
rebalancing	re bal ancing

Tokenization of Financial Text: A Concrete Example

Why this matters for finance

Numbers are split into arbitrary fragments, destroying their numerical meaning. Tickers and acronyms are decomposed into semantically meaningless pieces. This is one reason why LLMs struggle with arithmetic and precise entity matching — a theme we revisit in Lessons 3 and 5.

Stop Words: Helpful or Dangerous?

- ▶ Stop words are highly frequent terms that often carry limited discriminative power.
- ▶ Examples in English include words such as **the**, **and**, **to**, **of**.
- ▶ Removing them can reduce noise and dimensionality, but it may also delete relevant information.

There is no universal stop-word list

In sentiment analysis, removing **not** may completely alter the polarity of a sentence. In legal or financial documents, frequent function words may contribute to structure and meaning.

Stemming and Lemmatization

Stemming

A heuristic reduction of words to a crude root by removing prefixes or suffixes.

helpful → help

Stemming is fast, but the result is not always a valid word.

Lemmatization

A linguistically informed reduction to the canonical form (lemma):

was → be

It is usually cleaner, but it requires more linguistic knowledge.

Task dependence

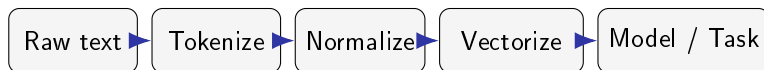
For retrieval, stemming or lemmatization may increase recall. For generation or high-fidelity parsing, aggressive normalization may remove useful distinctions.

Stemming and Lemmatization

- ▶ Lemmatization resolves words to their dictionary form (known as lemma) for which it requires detailed dictionaries in which the algorithm can look into and link words to their corresponding lemmas.
- ▶ For example, the words **running** , **runs** and **ran** are all forms of the word **run** , so **run** is the lemma of all the previous words.



A Minimal Classical Pipeline



What changes across paradigms?

The early stages remain recognizable even today, but the **vectorize** block changes radically when we move from sparse counts to embeddings and then to contextual representations.

Sparse Vector Representations of Text

The Representation Problem

Machine learning algorithms operate in a numerical feature space. Therefore, before we can train or compare models on text, we must transform documents into vectors.

Core question

Given a document d , what should the vector $\mathbf{x}(d)$ contain?

Three fundamental ideas dominated early NLP:

- ▶ represent a document through the words it contains;
- ▶ assign numerical weights to those words;
- ▶ define similarity or distance in the resulting vector space.

Bag-of-Words: The Simplest Encoding

In a **bag-of-words** model, each document is represented as a vector whose length equals the vocabulary size of the corpus.

Vocabulary

If the corpus vocabulary is

$$V = (w_1, w_2, \dots, w_{|V|}),$$

then each document d is mapped to a vector in $\mathbb{R}^{|V|}$.

What is ignored?

Bag-of-words ignores word order. It assumes that meaning is encoded primarily in vocabulary rather than in sequence.

Bag-of-Words: The Simplest Encoding

Suppose we have the three short documents:

d_1 : low interest rates usually mean low interest income

d_2 : banks lend money because money must create value

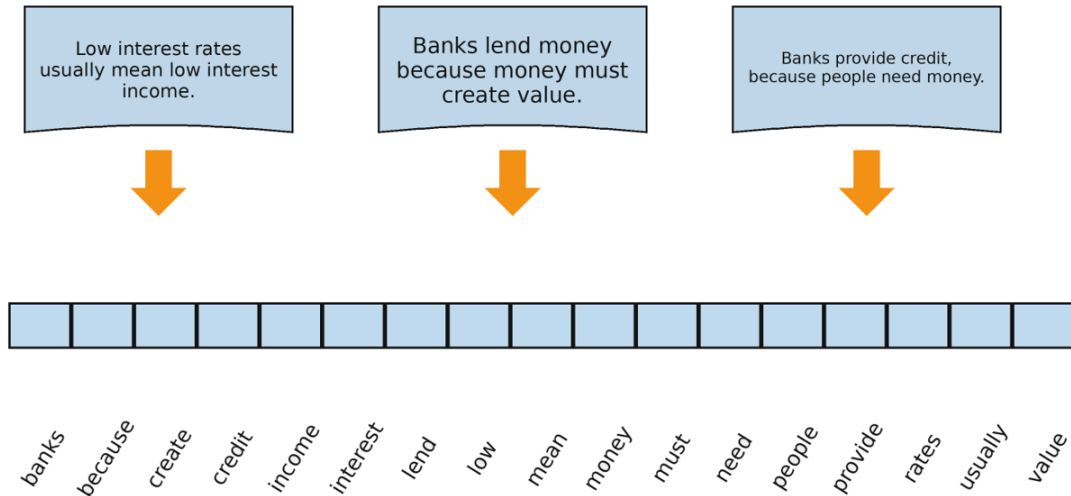
d_3 : banks provide credit because people need money

A possible vocabulary is:

$V = (\text{banks, because, create, credit, income, interest, lend, low, mean, money, must, need, people, provide, rates, usually, value})$

The corpus can then be represented as a **document-term matrix** where rows are documents and columns are vocabulary items.

Bag-of-Words: The Simplest Encoding



Count Vectors

The simplest bag-of-words encoding is the **count vector**:

$$\mathbf{x}(d) = (c_1, c_2, \dots, c_{|V|}),$$

where c_j is the number of occurrences of word w_j in document d .

Example

If

$$V = (\text{market}, \text{rises}, \text{falls}),$$

and the document is

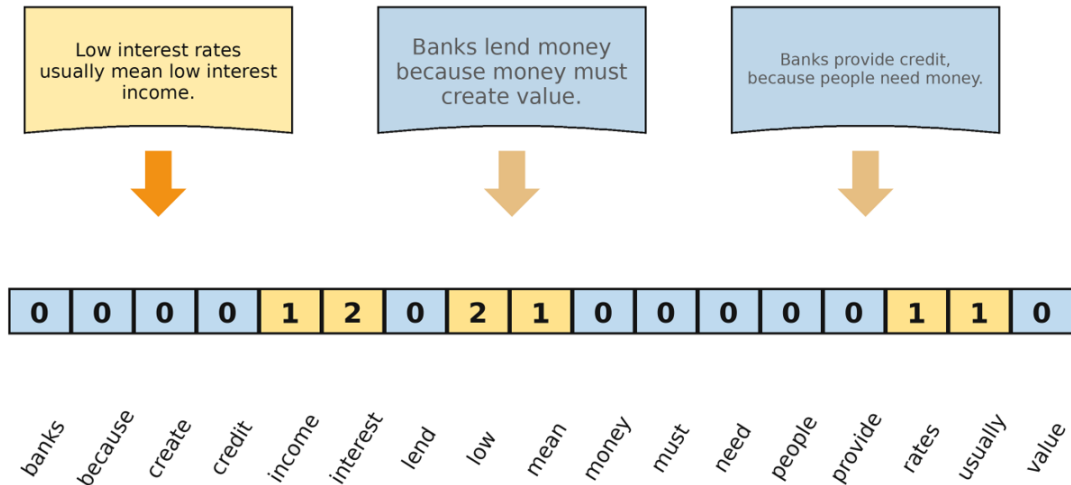
$$d = \text{market rises market},$$

then

$$\mathbf{x}(d) = (2, 1, 0).$$

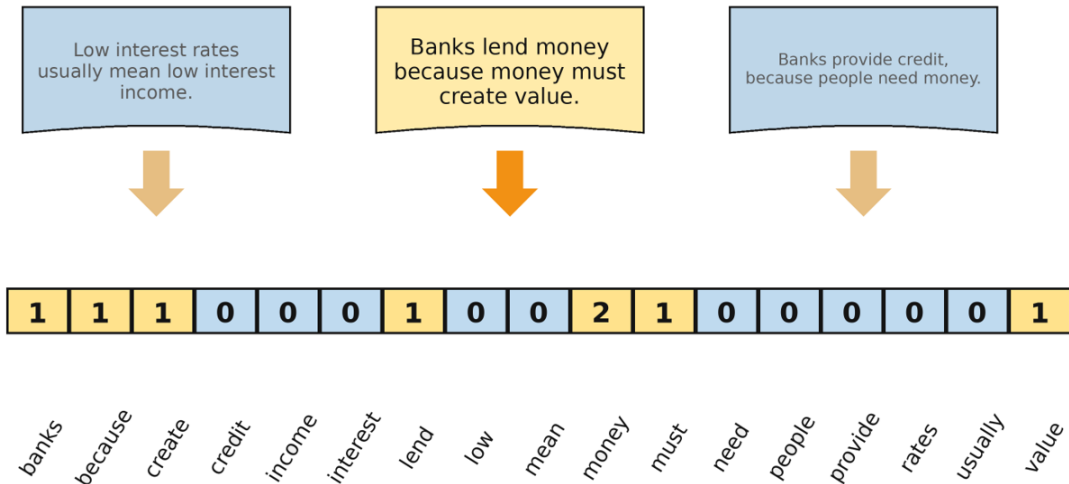
Bag-of-Words: The Simplest Encoding

Bag-of-Words representation - sentence 1



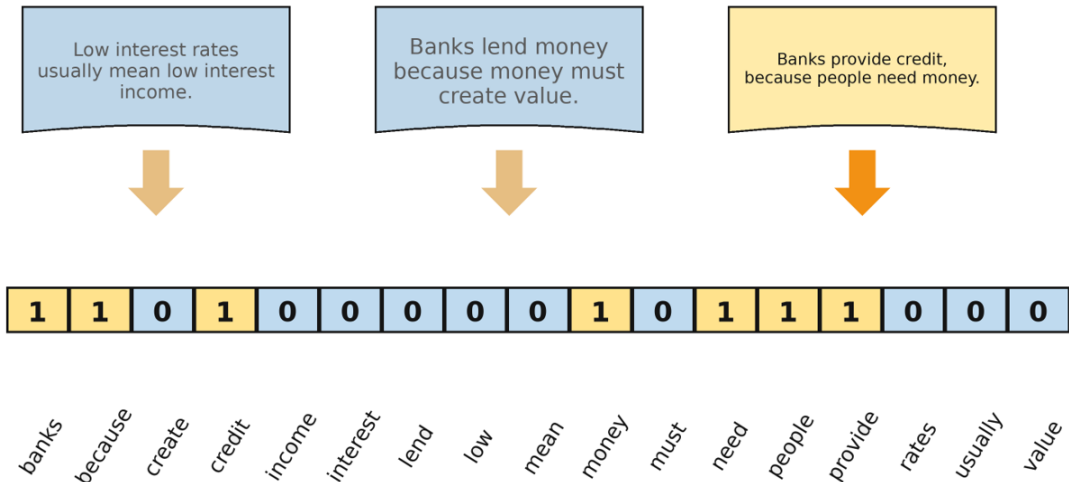
Bag-of-Words: The Simplest Encoding

Bag-of-Words representation - sentence 2



Bag-of-Words: The Simplest Encoding

Bag-of-Words representation - sentence 3



Beyond Single Words: N-grams

The bag-of-words model discards word order entirely. Two sentences with identical vocabulary but different arrangement receive identical representations.

A financial counterexample

the fund buy the bank $\neq$ the bank buy the fund

Identical unigram counts; very different meaning in practice.

An ***n*-gram** is a contiguous sequence of *n* tokens. Extending the feature space to sequences of length *n*:

<i>n</i>	Name	Financial example
1	unigram	rate, cut, yield
2	bigram	interest rate, yield curve
3	trigram	central bank policy

N-grams: Local Power and Global Sparsity

What n-grams capture

- ▶ Local word order and fixed phrases.
- ▶ Negation: no default, not profitable.
- ▶ Domain collocations: yield curve, central bank.
- ▶ Sentiment units: strong growth, sharp decline.

The sparsity cost

A bigram vocabulary has up to $|V|^2$ entries.
In practice:

- ▶ most bigrams are absent from any given document;
- ▶ the feature matrix becomes far sparser;
- ▶ memory and compute grow fast with n .

The fundamental tension

N-grams improve local context modelling but the vocabulary explosion and worsening sparsity limit their scalability. This is one of the motivations for learned dense representations.

One-Hot Encoding

A related but even simpler encoding is **one-hot** or **binary** encoding:

$$x_j(d) = \begin{cases} 1 & \text{if } w_j \text{ appears in } d, \\ 0 & \text{otherwise.} \end{cases}$$

Interpretation

Count vectors retain multiplicity; one-hot vectors only retain presence or absence.

Why we do not dwell on it today

One-hot is conceptually useful, but in modern NLP it is mostly a stepping stone toward richer representations.

Term Frequency (TF)

Raw counts are affected by document length. Longer documents tend to have larger counts even if they are not more informative.

A normalized alternative is **term frequency**:

$$\text{tf}(t, d) = \frac{\#(t \text{ in } d)}{\text{total number of tokens in } d}.$$

Interpretation

TF measures how important a term is *within a specific document*, relative to that document's length.

Inverse Document Frequency (IDF)

Some words are frequent in almost every document and therefore carry little discriminative power.

The **inverse document frequency** of term t in corpus D is:

$$\text{idf}(t, D) = \log \frac{N}{|\{d \in D : t \in d\}|},$$

where:

- ▶ N is the number of documents in the corpus;
- ▶ the denominator counts how many documents contain term t .

Rare terms have high IDF; ubiquitous terms have low IDF.

The most famous classical weighting scheme is:

$$\text{tfidf}(t, d, D) = \text{tf}(t, d) \cdot \text{idf}(t, D).$$

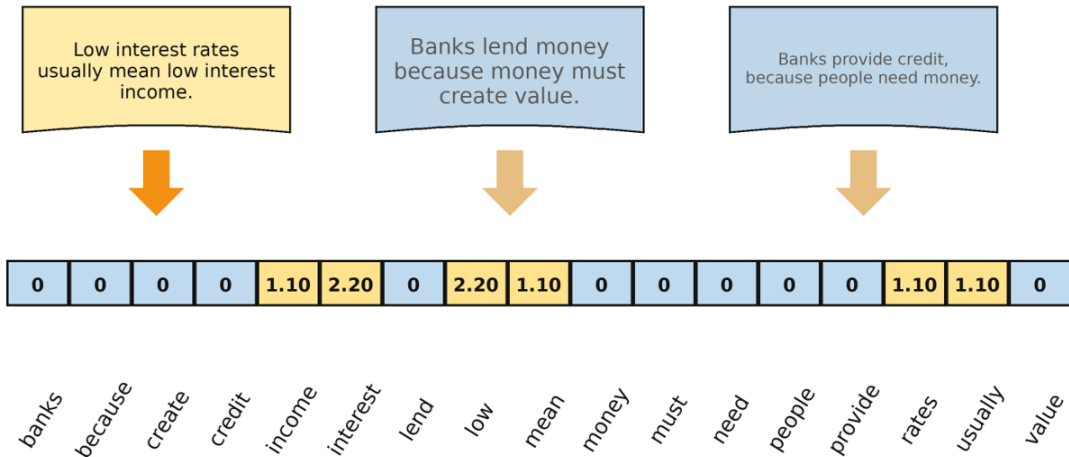
Why it works

TF-IDF rewards words that are frequent in a given document but not too frequent in the corpus as a whole.

- ▶ High TF alone may privilege long documents.
- ▶ High IDF alone may privilege rare but irrelevant words.
- ▶ Their product is a practical compromise for feature weighting.

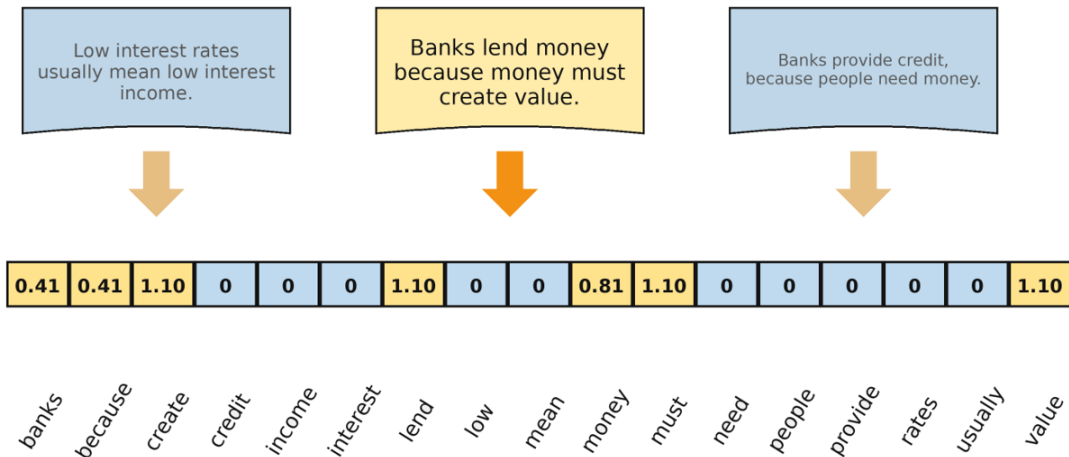
TF-IDF representation - sentence 1

TF = raw count, IDF = $\ln(N/df)$, $N = 3$



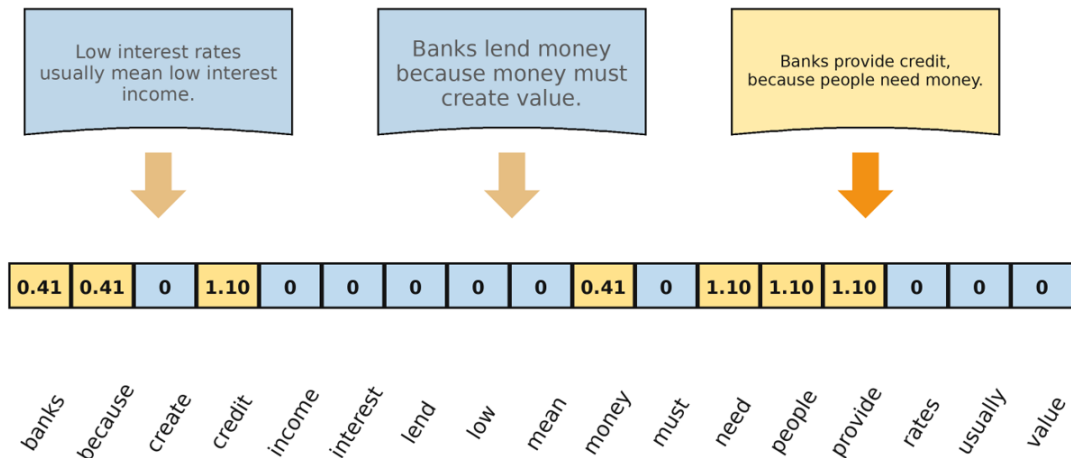
TF-IDF representation - sentence 2

TF = raw count, IDF = $\ln(N/df)$, $N = 3$



TF-IDF representation - sentence 3

TF = raw count, IDF = $\ln(N/df)$, $N = 3$



A Small TF-IDF Intuition

Suppose the term "**merger**" appears often in one specific financial news article, but only in a small fraction of the corpus.

Then:

- ▶ its **TF** is high in that article;
- ▶ its **IDF** is also relatively high in the corpus;
- ▶ hence its TF-IDF weight becomes large.

By contrast, a term such as "**the**" may have high TF in many documents, but its IDF is tiny because it appears everywhere.

A Worked Example: TF-IDF

Consider a corpus of $N = 3$ documents with vocabulary $V = \{\text{rate}, \text{cut}, \text{bond}\}$:

	Raw count			TF-IDF weight		
	rate	cut	bond	rate	cut	bond
d_1 : <i>rate cut rate</i>	2	1	0	$2 \times 0 = 0$	$1 \times 0.41 = 0.41$	0
d_2 : <i>bond rate bond</i>	1	0	2	$1 \times 0 = 0$	0	$2 \times 0.41 = 0.81$
d_3 : <i>rate bond cut</i>	1	1	1	$1 \times 0 = 0$	$1 \times 0.41 = 0.41$	$1 \times 0.41 = 0.41$

Using $\text{idf}(t) = \ln(N/\text{df}(t))$: rate appears in all 3 docs $\Rightarrow \ln(3/3) = 0$; cut and bond each in 2 docs $\Rightarrow \ln(3/2) \approx 0.41$. Note: rate vanishes entirely — a word present in every document has zero discriminative value under TF-IDF.

Measuring Similarity Between Documents

We now have a way to represent every document as a numerical vector (BoW counts or TF-IDF weights). A spontaneous question arises:

The core question

Given two document vectors x and y in $\mathbb{R}^{|V|}$, how do we quantify *how similar* they are?

This is not just an abstract question—it has immediate practical applications:

- ▶ **Search:** given a user query, find the most similar documents in a corpus.
- ▶ **Clustering:** group together documents that discuss similar topics.
- ▶ **Duplicate detection:** identify near-copies of financial filings or news articles.
- ▶ **Recommendation:** suggest articles similar to ones a user has already read.

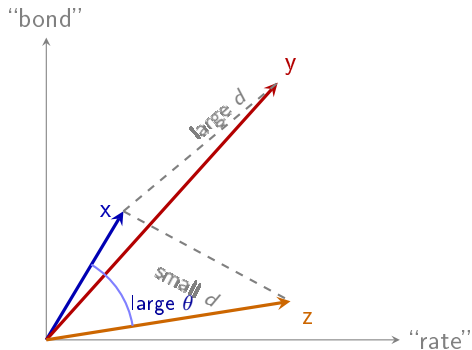
A First Attempt: Euclidean Distance

The most familiar distance measure is the Euclidean distance:

$$d(x, y) = \|x - y\| = \sqrt{\sum_{i=1}^{|V|} (x_i - y_i)^2}.$$

A First Attempt: Euclidean Distance

But it has a serious problem with text vectors.



The magnitude trap

x is a *short* article about bonds and rates.

y is a *long* article about the *same* topic — all TF-IDF weights are simply larger.

z is about a *different* topic entirely.

Euclidean distance says *x* is closer to *z* than to *y*, even though *x* and *y* discuss the same subject!

Cosine Similarity: Comparing Directions, Not Magnitudes

The fix is simple: ignore the length of the vectors and compare only their **directions**.

$$\cos(x, y) = \frac{x^\top y}{\|x\| \|y\|} = \frac{\sum_{i=1}^{|V|} x_i y_i}{\sqrt{\sum_i x_i^2} \sqrt{\sum_i y_i^2}}$$

Cosine Similarity: Comparing Directions, Not Magnitudes

Geometric intuition

Cosine similarity = the cosine of the **angle** θ between the two vectors.

$\cos \theta = 1 \Rightarrow$ identical direction

$\cos \theta = 0 \Rightarrow$ orthogonal (no overlap)

$\cos \theta = -1 \Rightarrow$ opposite directions

(For TF-IDF vectors, all weights ≥ 0 , so $\cos \theta \in [0, 1]$.)

Why this solves the problem

- ▶ Dividing by $\|x\|$ and $\|y\|$ **normalizes out** the effect of document length.
- ▶ A short article and a long article on the *same* topic will point in the *same* direction $\Rightarrow \cos \theta \approx 1$.
- ▶ Two articles on *different* topics will point in different directions $\Rightarrow \cos \theta \approx 0$.

Worked Example: Cosine Similarity

Using the TF-IDF vectors from the previous example (vocabulary: {rate, cut, bond}):

	rate	cut	bond
d_1	0	0.41	0
d_2	0	0	0.81
d_3	0	0.41	0.41

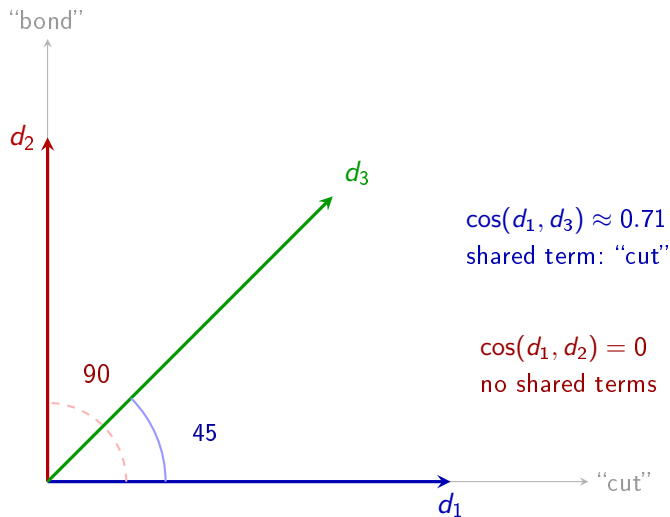
Compute $\cos(d_1, d_3)$:

$$\cos(d_1, d_3) = \frac{(0)(0) + (0.41)(0.41) + (0)(0.41)}{\sqrt{0^2 + 0.41^2 + 0^2} \sqrt{0^2 + 0.41^2 + 0.41^2}} = \frac{0.168}{0.41 \times 0.58} \approx 0.707$$

Compute $\cos(d_1, d_2)$:

$$\cos(d_1, d_2) = \frac{(0)(0) + (0.41)(0) + (0)(0.81)}{0.41 \times 0.81} = \frac{0}{0.33} = 0$$

A Simple Vector-Space Picture



From Feature Weighting to Document Retrieval

So far we have used TF-IDF as a **feature-weighting** scheme: given a fixed corpus, assign each term in each document a numeric weight that captures how *important* that term is for that document.

But TF-IDF vectors enable a second, very practical task:

Text Retrieval (a.k.a. Information Retrieval)

Given a collection of N documents and a short user **query** q (e.g. a few keywords), *rank* the documents from most relevant to least relevant with respect to q .

Examples in finance:

- ▶ Search “interest rate hike” across 50 000 analyst reports.
- ▶ Find the SEC filings most relevant to “credit default swap”.
- ▶ A RAG pipeline retrieves context passages before prompting an LLM.

How TF-IDF Retrieval Works

- 1 **Index Phase.** Represent every document d_j as a TF-IDF vector $v_j \in \mathbb{R}^{|V|}$.
- 2 **Query Phase.** Represent the query q as a (very sparse) TF-IDF vector v_q using the *same* vocabulary.
- 3 **Scoring.** Rank documents by similarity, typically **cosine similarity**:

$$\text{score}(q, d_j) = \cos(v_q, v_j) = \frac{v_q \cdot v_j}{\|v_q\| \|v_j\|}.$$

- 4 **Return** the top- k documents with the highest scores.

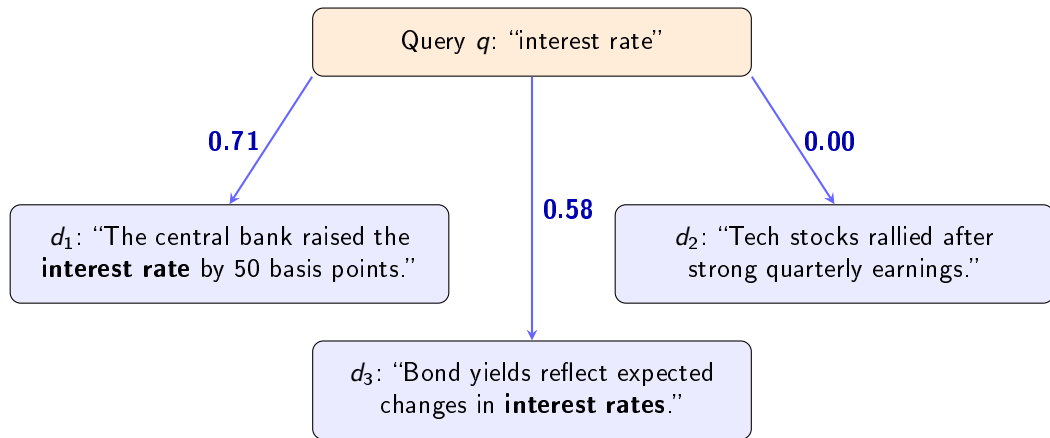
Key idea

The query is just another “document”—a very short one. We compare its vector against every document vector in the corpus and return the closest matches.

A Concrete Retrieval Example

Corpus: three financial news articles.

Query: “interest rate”



The Limits of TF-IDF for Retrieval

The retrieval mechanism we just described works, but in practice it produces **puzzling rankings**.

Observation 1 — Term repetition dominates the score

If a document repeats a query term many times (e.g. “inflation” appears 20 times), TF-IDF assigns it a much higher weight on that dimension than a document where the term appears only a few times — even when both documents are clearly *about* inflation. Repeating a word is not the same as being more relevant.

Observation 2 — Document length skews results

Longer documents naturally contain more tokens, and therefore accumulate higher raw TF counts. But a 50-page report that mentions “inflation” 100 times is not necessarily more relevant than a concise 1-page analysis that mentions it 5 times with greater focus.

Both observations suggest that **raw term frequency** is too crude a signal for ranking. But wait — didn't we introduce cosine similarity precisely to neutralize magnitude effects?

Wait — Doesn't Cosine Similarity Already Handle This?

After seeing these observations, a natural objection arises:

Question : “We showed that cosine similarity ignores vector magnitude. So shouldn't the normalization cancel out the effect of large TF counts and long documents?”

The answer involves a crucial distinction:

What cosine fixes

The **overall length** of the vector.

A 100-page document and a 1-page abstract with the *same term proportions* will have $\cos \theta = 1$ (identical direction, different magnitude).

What cosine does NOT fix

The **direction** of the vector.

If raw TF inflates one component disproportionately, the vector *tilts* toward that dimension. Cosine faithfully measures this tilted direction — it does not correct it.

Wait — Doesn't Cosine Similarity Already Handle This?

Bottom line: The problems we are about to describe are problems of the **TF-IDF weighting scheme** itself, not of the similarity metric. They distort the *direction* of document vectors, and cosine similarity *preserves* that distortion rather than correcting it.

Problem 1: Unbounded TF Distorts Vector Direction

Setup. Vocabulary $V = \{\text{inflation, tax, growth}\}$, and suppose $\text{IDF} = 1$ for all terms.

	inflation	tax	growth	cosine with q
Query q	1	0	0	—
Doc A (TF=20)	20	5	5	0.943
Doc B (TF=10)	10	5	5	0.816

Doc A mentions “inflation” twice as often as B , but both discuss tax and growth equally. Raw TF *tilts* A ’s vector more sharply toward the “inflation” axis, so cosine gives A a **16% higher score**.

The saturation problem

Is document A really more *relevant* than B just because it repeats “inflation” more often? In practice, relevance **saturates**: once a document clearly discusses inflation, additional mentions add **diminishing informational value**. Raw TF does not model this saturation — it lets the score keep rising with every repetition.

Problem 2: Document Length Distorts Term Proportions

Cosine similarity normalizes by vector *norm*, but document length introduces a subtler distortion.

The vocabulary-breadth effect

A **long** document (e.g. a 50-page annual report) covers many topics. Its TF-IDF vector has **many non-zero components** spread across hundreds of terms.

A **short** document (e.g. a 1-paragraph news flash) is tightly focused. Its vector has **few non-zero components**, concentrated on the topic at hand.

Effect on cosine similarity with a short query:

- ▶ The short document's weight is *concentrated* on the query terms $\Rightarrow$ high cosine.
- ▶ The long document's weight is *spread* across many dimensions $\Rightarrow$ the query-term components contribute a smaller share of the norm $\Rightarrow$ lower cosine, even if the document discusses the topic extensively.

Problem 2: Document Length Distorts Term Proportions

Paradox: cosine similarity can *penalize* long, comprehensive documents in favor of short ones — the opposite of the raw-TF bias, but equally undesirable. A good retrieval function should account for document length in a *controlled, tunable* way.

From These Observations to BM25

Problem	Root cause	BM25 solution
TF saturation	Raw TF grows without bound	Saturating function $\frac{tf \cdot (k_1 + 1)}{tf + k_1}$
Length bias	No explicit length control	Normalization factor $1 - b + b \frac{ d }{avgl}$

Historical context

These observations motivated **BM25** (*Best Match 25*), developed in the 1990s. It remains the standard sparse-retrieval baseline in 2025. BM25 does *not* use cosine similarity at all — it defines its own scoring function that incorporates saturation and length normalization directly into the formula.

BM25: Saturated Term Frequency and Length Normalization

For a query $Q = \{q_1, \dots, q_n\}$ and document d :

$$\text{BM25}(d, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{\text{tf}(q_i, d) \cdot (k_1 + 1)}{\text{tf}(q_i, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}} \right)}$$

Key parameters

- ▶ $k_1 \in [1.2, 2.0]$: **TF saturation**.
As $\text{tf} \rightarrow \infty$, the fraction approaches $k_1 + 1$.
- ▶ $b \approx 0.75$: **length normalization**.
- ▶ $|d|$: document length; avgdl: corpus average.

Special cases

- ▶ $b = 0$: no length normalization.
- ▶ $k_1 = 0$: degenerates to pure IDF.

Standard defaults ($k_1=1.2$, $b=0.75$) work well across domains without tuning.

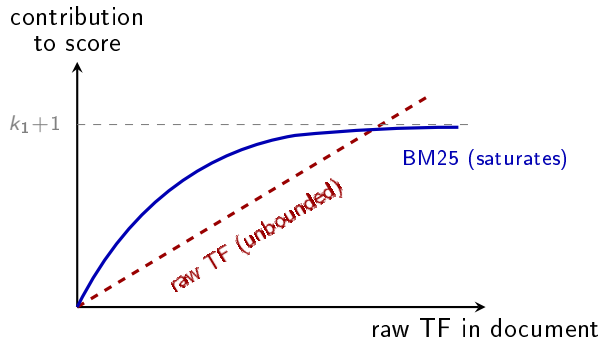
BM25: Fixing Both Problems in One Formula

$$\text{BM25}(d, Q) = \sum_i \underbrace{\text{IDF}(q_i)}_{\text{term rarity}} \cdot \frac{\text{tf} \cdot (k_1 + 1)}{\underbrace{\text{tf} + k_1 \cdot \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)}_{\text{length adjustment}}}_{\text{saturating TF}}$$

BM25: Fixing Both Problems in One Formula

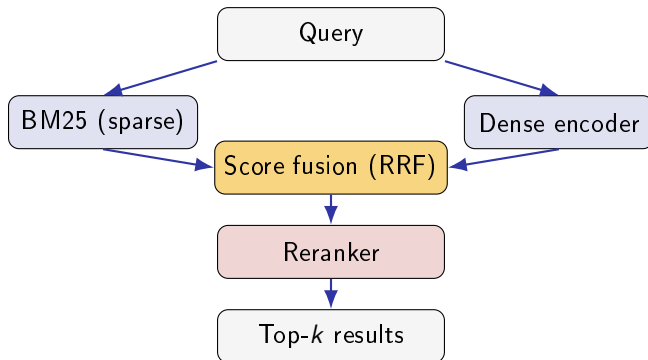
Reading the formula:

- **IDF**: same role as in TF-IDF — rare terms count more.
- **Saturating TF**: as tf grows, the fraction approaches a ceiling (k_1+1) instead of growing forever.
- **Length adjustment**: if $|d|$ is longer than the corpus average, the denominator grows, *dampening* the score.



Why BM25 survives in 2025

- ▶ No training; fast inference.
- ▶ Strong on exact-match and rare terms.
- ▶ Reproducible, well-understood baseline.
- ▶ Runs on standard inverted indexes.



Forward reference – Lesson 4

RAG pipelines run BM25 in parallel with a dense encoder. Outputs are fused via RRF and reranked before being passed to the LLM.

Sparse Vectors and High Dimensionality

Bag-of-words and TF-IDF vectors are usually:

- ▶ **high-dimensional**: vocabulary size can be tens of thousands or more;
- ▶ **sparse**: each document contains only a small subset of the vocabulary;
- ▶ **interpretable**: each dimension corresponds to a known word.

Strength

Sparse vectors are simple, transparent, and often surprisingly effective.

Weakness

They do not directly encode semantic proximity between different words such as car and automobile.

Jupyter Lab

In the next part of the lecture, we will translate the main ideas into code and build a complete workflow in Python.

Focus of the lab: SEC filings, text preprocessing, TF-IDF, and document similarity
notebook_01.ipynb

From Sparse Vectors to Word Embeddings

Why a New Representation Was Needed

Two sentences may be semantically close while sharing very few words.

For example:

s_1 : The company announced a merger.

s_2 : The firm disclosed an acquisition deal.

A sparse lexical model may understate their similarity because words such as company/firm and merger/acquisition occupy different coordinates.

Goal

We want a representation where semantically similar words and documents can be geometrically close.

What Is a Word Embedding?

A **word embedding** maps each word to a dense vector in a low-dimensional continuous space:

$$w \longmapsto \mathbf{e}(w) \in \mathbb{R}^k,$$

where k is typically much smaller than the vocabulary size.

Key idea

Meaning is represented not by an isolated symbol but by the position of a word inside a geometric space learned from data.

Nearby words in that space tend to share similar distributional behavior.

The Distributional Hypothesis

A foundational intuition behind embeddings is the **distributional hypothesis**:

Words that occur in similar contexts tend to have similar meanings.

- ▶ If `bank`, `lender`, and `institution` often appear near similar words, their vectors should become similar.
- ▶ If `river` and `shore` appear in another type of local environment, they should form another semantic neighborhood.

The Company Words Keep

Consider a small financial corpus:

- 1 The **bank** approved a new **loan** for the client.
- 2 The **lender** granted a large **loan** to the firm.
- 3 The **bank** increased its **interest** on deposits.
- 4 The **lender** raised the **interest** rate on mortgages.
- 5 The **river** flows under the old stone **bridge**.
- 6 The **river** flooded the **bridge** after heavy rain.

Look at the **neighbors** of each word. What words tend to appear near *bank*? And near *lender*? And near *river*?

Observing the Context Windows

If we look at which words appear within a window of ± 2 positions around each target word, we find:

bank

approved, loan,
new, client,
increased, interest,
its, deposits

lender

granted, loan,
large, firm,
raised, interest,
rate, mortgages

river

flows, bridge,
under, old,
flooded, bridge,
after, heavy

Key observation

bank and **lender** share context words (*loan, interest*) — their neighborhoods overlap.
river shares *none* of these contexts. It lives in a completely different linguistic environment.

From Contexts to a Co-occurrence Table

We can formalize this intuition. Count how many times each word appears *near* each other word across the whole corpus:

	loan	interest	bridge	approved	granted	flooded
bank	2	1	0	1	0	0
lender	2	1	0	0	1	0
river	0	0	2	0	0	1

From Contexts to a Co-occurrence Table

Each row is already a vector representation!

The row for *bank* = (2, 1, 0, 1, 0, 0)

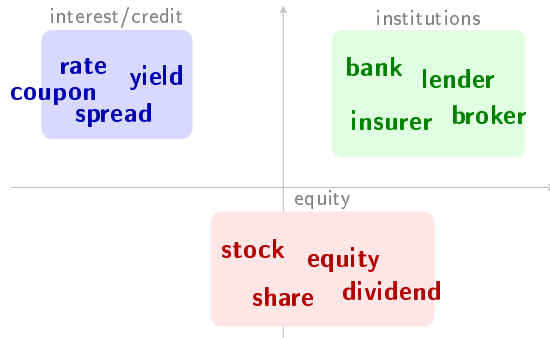
The row for *lender* = (2, 1, 0, 0, 1, 0)

The row for *river* = (0, 0, 2, 0, 0, 1)

The rows for **bank** and **lender** point in a *similar direction*. The row for **river** points *elsewhere*.

Semantic similarity has become geometric proximity.

The Semantic Map Analogy



So What Do Word2Vec and GloVe Actually Do?

The co-occurrence table already captures semantic proximity. But it has a practical problem:

The curse of dimensions

In a real corpus, the vocabulary may contain $|V| = 50\,000$ words.

Each row of the co-occurrence table is a vector in $\mathbb{R}^{50\,000}$ — **extremely sparse** and expensive to store and compare.

The key idea behind embedding models

Compress the co-occurrence information into a much **smaller, dense** vector — typically $k = 100\text{--}300$ dimensions.

Co-occurrence row $\longrightarrow$ Dense embedding
 $x \in \mathbb{R}^{50\,000}$ (sparse) $e \in \mathbb{R}^{300}$ (dense)

The compression preserves the essential structure: words that were close in the big sparse space **remain close** in the small dense space.

Dense vs Sparse Representations

Sparse lexical vectors

- ▶ dimension tied to vocabulary size
- ▶ mostly zeros
- ▶ highly interpretable
- ▶ weak semantic generalization

Dense embeddings

- ▶ low-dimensional continuous space
- ▶ almost no zeros
- ▶ less directly interpretable
- ▶ stronger semantic generalization

Trade-off

The field moves from hand-readable coordinates to learned coordinates that are much more useful for generalization.

In an idealized embedding space, semantic regularities appear as geometric relations. For example, one may observe patterns of the form:

$$\mathbf{e}(\text{king}) - \mathbf{e}(\text{man}) + \mathbf{e}(\text{woman}) \approx \mathbf{e}(\text{queen}).$$

Interpretation

The exact analogy should never be taken too literally, but it illustrates a central idea: distributed representations can encode regular semantic and syntactic structure.

Static Embeddings and the Role of Context

Static embeddings assign **one vector per word type**. Therefore:

$$\mathbf{e}(\text{bank}) = \text{same vector in all occurrences.}$$

This is both powerful and limited

- ▶ Powerful: the model learns semantic neighborhoods from co-occurrence patterns.
- ▶ Limited: the same word may have different meanings in different contexts.

This limitation will later motivate contextual embeddings and transformers.

Word2Vec learns embeddings by solving a local prediction problem.

Two main variants are used:

- ▶ **CBOW** (Continuous Bag of Words): predict the central word from surrounding context.
- ▶ **Skip-Gram**: predict surrounding words from the central word.

Why this matters

The network is not trained to “understand semantics” directly. Semantics emerges because predicting context forces the model to organize words according to their usage environments.

Suppose the context window around a target word is:

(the, central, _____, raised, rates)

CBOW uses the surrounding words to predict the missing central token:

(the, central, raised, rates) $\rightarrow$ bank

Intuition

Words with similar neighboring environments will tend to receive similar embeddings.

Skip-Gram reverses the direction:

`bank → (central, raised, rates, ...)`

Why it is interesting

A word vector becomes useful if it is informative about the kinds of contexts the word tends to generate.

- ▶ Frequent words produce many training events.
- ▶ Nearby words in vector space tend to predict similar local environments.

The Question We Have Not Yet Answered

We have established *what* a word embedding is:

$$w \mapsto e(w) \in \mathbb{R}^k \quad \text{with } k \ll |V|$$

And we have seen *why* it is useful: words with similar contexts receive similar vectors.

The missing piece

How does the training procedure actually produce these vectors? Where do the numbers come from?

We will answer this question using only elementary ideas — no neural network formalism is required. The full machinery (backpropagation, loss functions, optimizers) will be introduced in Lesson 2.

Step 1: The Lookup Table

Imagine a large spreadsheet with one row per word and k columns of real numbers. This spreadsheet *is* the embedding.

word	e_1	e_2	e_3	$\dots$	e_k
bank	0.31	0.72	0.18	$\dots$	0.54
lender	0.28	0.65	0.21	$\dots$	0.49
river	0.61	0.44	0.33	$\dots$	0.12
loan	0.35	0.58	0.25	$\dots$	0.61

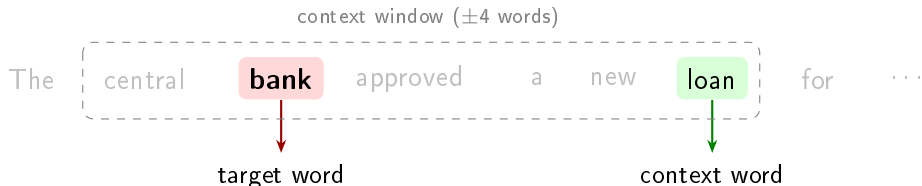
} $e(\text{bank}) \in \mathbb{R}^k$

Key point

Nobody designs these numbers by hand. At the start of training, every entry is a **small random number**. The training procedure will adjust them until they encode useful semantic information.

Step 2: The Training Game (Skip-Gram)

Word2Vec learns by playing a simple guessing game, repeated billions of times over the corpus.



The game: given the target word **bank**, can the model predict that **loan** is one of its neighbors?

The model's *only* resource for making this prediction is the lookup table — the embedding vectors.

Step 3: Making a Prediction

How does the model decide whether *loan* is a plausible neighbor of *bank*? With a single operation:

$$\text{score}(\text{bank}, \text{loan}) = \mathbf{e}(\text{bank}) \cdot \mathbf{e}(\text{loan}) = \sum_{j=1}^k e_j(\text{bank}) e_j(\text{loan})$$

- ▶ If the **dot product is large** $\Rightarrow$ the model predicts that *loan* is a likely neighbor of *bank*.
- ▶ If the **dot product is small or negative** $\Rightarrow$ the model predicts that the two words are unlikely to appear together.

What matters

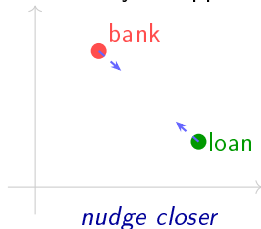
The dot product is large when the two vectors point in a similar direction. So the model is comparing **directions in embedding space** — exactly the same geometric idea as cosine similarity.

Step 4: Adjusting the Table

After each prediction, the model checks the answer and makes a small correction. The rule is simple in principle:

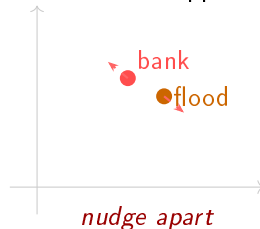
Case A: correct pair

bank and *loan* really do appear together.



Case B: incorrect pair

bank and *flood* do not appear together.



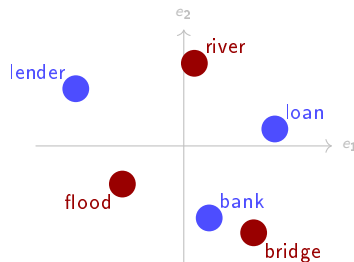
That is the entire mechanism

Pull together vectors of words that co-occur. Push apart vectors of words that do not. Repeat billions of times. The final positions encode distributional similarity.

A Toy Example: Before Training

Suppose our vocabulary has six words and we use $k = 2$ dimensions (so we can draw them).

word	e_1	e_2
bank	0.12	-0.34
lender	-0.51	0.27
loan	0.43	0.08
river	0.05	0.39
flood	-0.29	-0.18
bridge	0.33	-0.41



All entries are **small random numbers**.
No structure at all.

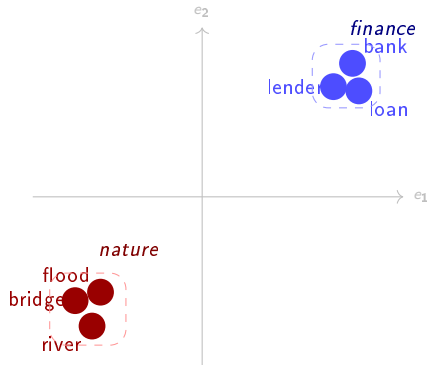
Before training: scattered randomly.

A Toy Example: After Training

After billions of such adjustments over the entire corpus:

word	e_1	e_2
bank	0.71	0.63
lender	0.62	0.52
loan	0.74	0.50
river	-0.52	-0.61
flood	-0.48	-0.45
bridge	-0.60	-0.49

Rows that were random now show **clear clusters**.



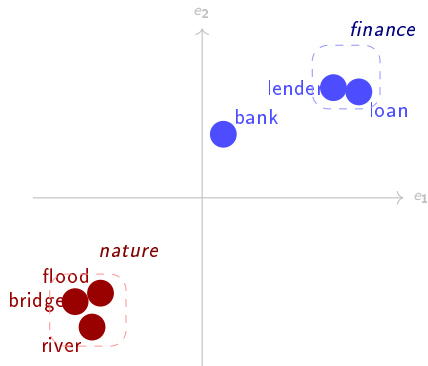
After training: semantic clusters emerge.

A Toy Example: After Training (more Realistic!)

After billions of such adjustments over the entire corpus:

word	e_1	e_2
bank	0.71	0.63
lender	0.62	0.52
loan	0.74	0.50
river	-0.52	-0.61
flood	-0.48	-0.45
bridge	-0.60	-0.49

Rows that were random now show **clear clusters**.



After training: semantic clusters emerge.

GloVe: Another Route to Dense Representations

GloVe stands for **Global Vectors for Word Representation**. It combines two ideas:

- ▶ local co-occurrence information, as in prediction-based models;
- ▶ global corpus statistics, summarized by a co-occurrence matrix.

Co-occurrence matrix

Entry X_{ij} records how often word w_j appears in the context of word w_i .

Why Co-occurrence Matters

If two words tend to appear in similar neighborhoods, their co-occurrence profiles should be similar.

Example

If bank frequently co-occurs with loan, interest, and deposit, while riverbank co-occurs with water, shore, and flood, then a useful representation should preserve this distinction.

GloVe starts from a large table of word co-occurrences. It then learns dense vectors that preserve the main statistical relationships in that table. In this way, words that appear in similar linguistic environments become close in the embedding space.

Sparse to Dense: What We Gain and What We Lose

What we gain

- ▶ semantic smoothness
- ▶ generalization across vocabulary
- ▶ useful initialization for downstream tasks
- ▶ better handling of synonymy and analogy

What we lose

- ▶ easy interpretability of each dimension
- ▶ direct lexical transparency
- ▶ a simple one-feature-one-word reading

From Word Vectors to Document Vectors

Word embeddings assign a vector to each *word*. But many downstream tasks — retrieval, classification, clustering — require a vector for an entire *document*.

The simplest bridge: mean pooling

Given a document $d = (w_1, \dots, w_L)$ with word embeddings $\mathbf{e}(w_i)$:

$$\mathbf{d}_{\text{mean}} = \frac{1}{L} \sum_{i=1}^L \mathbf{e}(w_i).$$

From Word Vectors to Document Vectors

Word embeddings assign a vector to each *word*. But many downstream tasks — retrieval, classification, clustering — require a vector for an entire *document*.

Weighted averaging

A common refinement weights each word by its TF-IDF score:

$$\mathbf{d}_{\text{tfidf}} = \frac{\sum_{i=1}^L \text{tfidf}(w_i, d, D) \mathbf{e}(w_i)}{\sum_{i=1}^L \text{tfidf}(w_i, d, D)}.$$

This down-weights frequent, uninformative words and up-weights distinctive terms — combining the strengths of sparse weighting with dense geometry.

Document Embeddings: Beyond Simple Averaging

Doc2Vec (Le & Mikolov, 2014)

Extends Word2Vec by introducing a learnable **paragraph vector** that participates in the prediction task alongside word embeddings. Each document receives its own trainable vector.

SIF embeddings (Arora et al., 2017)

A theoretically motivated scheme:

- 1 compute weighted average using smooth inverse frequency weights;
- 2 remove the first principal component (common discourse vector).

Surprisingly competitive with more complex models.

Modern sentence encoders

Models like Sentence-BERT (2019) and the current generation of embedding models (e5, GTE, nomic-embed) produce document-level dense vectors directly from transformer encoders. These are the backbone of modern semantic search.

⇒ **Lesson 2** will explain the neural foundations;

⇒ **Lesson 4** will show how these vectors drive retrieval and RAG.

The Complementarity of Sparse and Dense

Now that we have seen both sparse and dense representations, we can appreciate their **complementary** failure modes:

Sparse fails at semantics

monetary tightening does not match a document containing only interest rate hike, although they are equivalent in meaning. Sparse methods require lexical overlap.

Dense fails at precision

An embedding model may rank a document about Apple Inc. highly for a query about apple orchards because their representations overlap in geometric space.

Hybrid retrieval

Combine sparse and dense signals. Each component compensates for the other's blind spot. This is the architecture underlying most production-grade search and RAG systems in 2025. Score fusion is typically performed via **Reciprocal Rank Fusion (RRF)**, which combines the two ranked lists by position rather than by raw score – no calibration required.

The Importance of Context

Static embeddings are a major improvement over sparse lexical vectors, but they still assign a single vector to each word type.

Problem

The word current in

- ▶ current news
- ▶ electric current

should not have exactly the same representation in both cases.

This is where the field begins to move toward **contextual embeddings**, which will be the topic of the next lesson.

A Compact Comparison of Representational Stages

Stage	Unit	Representation	Main limitation
BoW / TF-IDF	document	sparse lexical weights	ignores semantics and order
Word2Vec / GloVe	word type	dense static vector	ignores context; one vector per type
Doc averaging / Doc2Vec	document	dense static (pooled)	still no context sensitivity
Contextual models	token in context	context-dependent vector	more complex and less transparent

Forward reference – byte-level models

The most recent architectures (ByT5, MegaByte, byte-latent transformers) explore **eliminating subword tokenization entirely**, operating directly on raw bytes. This may reshape how financial text, numbers, and tickers enter language models — a theme we revisit in Lesson 3.

Conclusion and Transition

What We Have Learned Today

- ▶ Text must be transformed into numerical structure before we can learn from it.
- ▶ Classical NLP begins with corpora, preprocessing, and sparse vector representations.
- ▶ Bag-of-words, TF-IDF, and BM25 remain important because they make the feature space explicit, interpretable, and surprisingly competitive.
- ▶ Word embeddings replace sparse lexical coordinates with dense distributed representations; document-level vectors can be derived by pooling or learning.
- ▶ Sparse and dense methods have complementary failure modes; modern retrieval systems combine both.
- ▶ Static embeddings improve semantic generalization, but they do not solve context dependence — the topic of our next lesson.

Main Takeaways

Takeaway 1

A representation is never neutral: every representation encodes assumptions about what matters in language.

Takeaway 2

Sparse lexical models are still useful, but they cannot fully capture semantic proximity or contextual meaning.

Takeaway 3

The move from counts to embeddings is not a cosmetic improvement: it is the beginning of a new representational regime in NLP.

Looking Ahead to Lesson 2

In the next lesson, we will deepen the transition that begins today.

We will study:

- ▶ why neural networks matter for language representations;
- ▶ how a model can learn features instead of relying on manual feature engineering;
- ▶ how contextual embeddings emerge from self-attention and transformer architectures.

Bridge to the next step

Today we learned how to turn text into vectors. Next time we will ask a deeper question: **how can a model learn the right representation by itself, and how does context change the meaning of a token?**

Appendix

Appendix 1 — The Statistical Structure of Language: Zipf's Law

An empirical regularity that shapes every natural-language corpus:

Zipf's law (G. K. Zipf, 1949)

If words are ranked by frequency, the frequency of the r -th most common word satisfies:

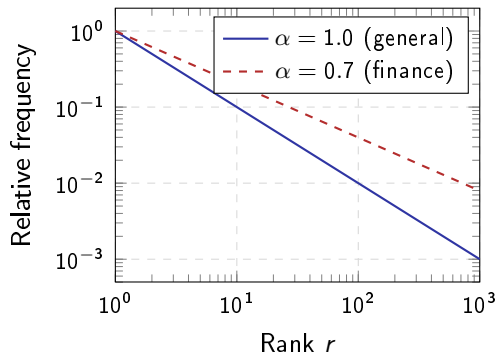
$$\text{freq}(r) \propto \frac{1}{r^\alpha}, \quad \alpha \approx 1.$$

Immediate consequences:

- ▶ A handful of words (the, of, and, ...) accounts for roughly **50 %** of all tokens in a typical English corpus.
- ▶ A large fraction of the vocabulary consists of **hapax legomena**: words appearing exactly once.
- ▶ Raw count vectors are dominated by high-frequency, low-information words.

Appendix 1 — Zipf's Law: Why It Matters for Text Representation

Power-law rank–frequency distribution



Connection to IDF

IDF is a statistical correction for Zipf's law. It penalizes the terms that cluster at the top of the frequency distribution – ubiquitous but discriminatively empty.

Financial corpora

Domain terms (EBITDA, haircut, collateral call) are rare in general language but informationally dense. They receive high IDF weights – and hence high TF-IDF scores when prominent in a document.

Appendix 2 — The Score Fusion Problem

In a hybrid retrieval pipeline, two (or more) independent systems each return a **ranked list** of documents for the same query.

BM25 (sparse)

Rank	Doc	Score
1	d_3	18.7
2	d_1	12.4
3	d_5	9.1
4	d_2	6.3
5	d_4	2.8

Dense encoder

Rank	Doc	Score
1	d_1	0.93
2	d_3	0.88
3	d_7	0.81
4	d_5	0.79
5	d_6	0.72

The problem

How do we merge these two lists into a *single* ranking?

We **cannot** simply average the scores: BM25 produces unbounded values (18.7, 12.4, ...) while the dense encoder produces cosine similarities in $[0, 1]$. The two scales are **incommensurable**.

Appendix 2 — Reciprocal Rank Fusion: The Idea

Key insight: ignore the raw scores entirely. Use only the **position** (rank) of each document in each list.

RRF formula

Given R ranked lists, the fused score of document d is:

$$\text{RRF}(d) = \sum_{r=1}^R \frac{1}{k + \text{rank}_r(d)}$$

where $\text{rank}_r(d)$ is the position of d in the r -th list (1 = top), and k is a smoothing constant (typically $k = 60$).

Appendix 2 — Reciprocal Rank Fusion: The Idea

Why this works:

- ▶ A document ranked **high** in a list receives a **large** reciprocal ($\frac{1}{61} \approx 0.0164$ for rank 1).
- ▶ A document ranked **low** receives a **small** reciprocal ($\frac{1}{70} \approx 0.0143$ for rank 10).
- ▶ A document that does **not appear** in a list is treated as having rank $= \infty$, contributing ≈ 0 .
- ▶ The constant $k = 60$ prevents the top-ranked document from dominating excessively: the difference between rank 1 and rank 2 is $\frac{1}{61} - \frac{1}{62} \approx 0.0003$, not a factor of 2.

Appendix 2 — RRF: Worked Example ($k = 60$)

Using the two lists from the previous slide:

Doc	BM25 rank	$\frac{1}{60+\text{rank}}$	Dense rank	$\frac{1}{60+\text{rank}}$	RRF score
d_1	2	$\frac{1}{62} = .0161$	1	$\frac{1}{61} = .0164$	.0325
d_3	1	$\frac{1}{61} = .0164$	2	$\frac{1}{62} = .0161$	.0325
d_5	3	$\frac{1}{63} = .0159$	4	$\frac{1}{64} = .0156$	.0315
d_7	—	≈ 0	3	$\frac{1}{63} = .0159$	.0159
d_2	4	$\frac{1}{64} = .0156$	—	≈ 0	.0156
d_6	—	≈ 0	5	$\frac{1}{65} = .0154$	.0154
d_4	5	$\frac{1}{65} = .0154$	—	≈ 0	.0154

Appendix 2 — RRF: Worked Example ($k = 60$)

Fused ranking

$$d_1 \approx d_3 \succ d_5 \succ d_7 \succ d_2 \approx d_6 \approx d_4$$

Documents d_1 and d_3 tie because they hold symmetric positions. d_5 ranks third because it appears in **both** lists, while d_7 , d_2 , d_6 , d_4 each appear in only one.

Appendix 2 — Why RRF Is the Default Fusion Method

Advantages

- ▶ **No calibration needed.** Works with any combination of rankers regardless of their score scales.
- ▶ **No training data.** A single hyperparameter (k) with a robust default ($k = 60$).
- ▶ **Rewards consensus.** Documents ranked highly by *multiple* systems float to the top.
- ▶ **Extensible.** Adding a third ranker is trivial — just add another term to the sum.

Limitations

- ▶ **Ignores score magnitude.** A document ranked 1st with score 0.99 and one ranked 1st with score 0.51 are treated identically.
- ▶ **Equal weight to all rankers.** If one ranker is much better than the other, RRF does not know this.
- ▶ **Truncation matters.** If a ranker returns only top-100 documents, anything beyond that cutoff is invisible.